

Input/Output

- **Unix Philosophy:** "Everything is a file." Whether it's a disk file, a keyboard, or a network socket, you interact with it using the same set of system calls.
- **Windows Philosophy:** Everything is an Object managed by the Object Manager. I/O is handled via Handles to these objects, often using the I/O Request Packet (IRP) model.

Unix Files

- A Unix *file* is a sequence of m bytes:
 - $B_0, B_1, \dots, B_k, \dots, B_{m-1}$
- All I/O devices are represented as files:
 - `/dev/sda2` (`/usr` disk partition)
 - `/dev/tty2` (terminal)
- Even the kernel is represented as a file:
 - `/dev/kmem` (kernel memory image)
 - `/proc` (kernel data structures)

Unix File Types

■ Regular file

- File containing user/app data (binary, text, whatever)
- OS does not know anything about the format
 - other than “sequence of bytes”, akin to main memory

■ Directory file

- A file that contains the names and locations of other files

■ Character special and block special files

- Terminals (character special) and disks (block special)

■ FIFO (named pipe)

- A file type used for inter-process communication

■ Socket

- A file type used for network communication between processes

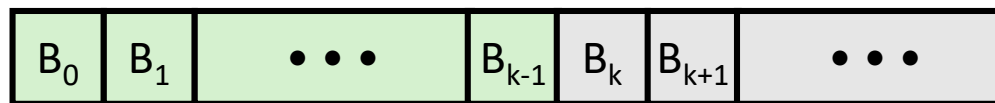
Unix I/O

■ Key Features

- Elegant mapping of files to devices allows kernel to export simple interface called Unix I/O
- Important idea: All input and output is handled in a consistent and uniform way

■ Basic Unix I/O operations (system calls):

- Opening and closing files
 - `open()` and `close()`
- Reading and writing a file
 - `read()` and `write()`
- Changing the *current file position* (seek)
 - indicates next offset into file to read or write
 - `lseek()`



Current file position = k

Core System Calls: Open, Read, Write, Close

Feature	Unix System Call	Windows API (Win32)
Open	<code>open(path, flags, mode)</code>	<code>CreateFile(...)</code> (used for both creation and opening)
Close	<code>close(fd)</code>	<code>CloseHandle(hObject)</code>
Read	<code>read(fd, buf, count)</code>	<code>ReadFile(hFile, lpBuffer, ...)</code>
Write	<code>write(fd, buf, count)</code>	<code>WriteFile(hFile, lpBuffer, ...)</code>

■ **Short Counts.** In Unix, read and write might return fewer bytes than requested (a short count). This isn't an error; it happens due to:

- Encountering **EOF** (End of File).
- Reading from a terminal or network socket.
- Interruption by a signal.
- Windows handles this similarly but often relies on explicit "bytes transferred" pointers in the API call.

Opening Files

- Opening a file informs the kernel that you are getting ready to access that file

```
int fd;    /* file descriptor */  
  
if ((fd = open("/etc/hosts", O_RDONLY)) < 0) {  
    perror("open");  
    exit(1);  
}
```

- Returns a small identifying integer *file descriptor*
 - `fd == -1` indicates that an error occurred
- Each process created by a Unix shell begins life with three open files associated with a terminal:
 - 0: standard input
 - 1: standard output
 - 2: standard error

Closing Files

- Closing a file informs the kernel that you are finished accessing that file

```
int fd;      /* file descriptor */
int retval;  /* return value */

if ((retval = close(fd)) < 0) {
    perror("close");
    exit(1);
}
```

- Closing an already closed file is a recipe for disaster in threaded programs (more on this later)
- Moral: Always check return codes, even for seemingly benign functions such as `close()`

Reading Files

- Reading a file copies bytes from the current file position to memory, and then updates file position

```
char buf[512];
int fd;          /* file descriptor */
int nbytes;      /* number of bytes read */

/* Open file fd ... */
/* Then read up to 512 bytes from file fd */
if ((nbytes = read(fd, buf, sizeof(buf))) < 0) {
    perror("read");
    exit(1);
}
```

- Returns number of bytes read from file `fd` into `buf`
 - Return type `ssize_t` is signed integer
 - `nbytes < 0` indicates that an error occurred
 - **Short counts** (`nbytes < sizeof(buf)`) are possible and are not errors!

Writing Files

- Writing a file copies bytes from memory to the current file position, and then updates current file position

```
char buf[512];
int fd;          /* file descriptor */
int nbytes;      /* number of bytes read */

/* Open the file fd ... */
/* Then write up to 512 bytes from buf to file fd */
if ((nbytes = write(fd, buf, sizeof(buf))) < 0) {
    perror("write");
    exit(1);
}
```

- Returns number of bytes written from `buf` to file `fd`
 - `nbytes < 0` indicates that an error occurred
 - As with reads, short counts are possible and are not errors!

Metadata and Representation

■ File Metadata

- **Unix:** Uses the `stat` structure. You access it via `stat()`, `fstat()`, or `lstat()`. It includes the **inode** number, size, permissions, and timestamps.
- **Windows:** Uses `GetFileInformationByHandle`. Metadata includes file attributes (Hidden, System, Archive), which are more complex than Unix's simple bitmask permissions.

• How the Kernel Represents Open Files

- **Descriptor Table (Unix):** Each process has its own table of pointers to the open file table.
- **Open File Table:** A system-wide table containing the current file position (offset), reference count, and a pointer to the v-node/i-node.
- **v-node/i-node Table:** Contains the actual file metadata and location on disk.
- **Windows** uses a similar "**Handle Table**" for each process, which points to executive file objects in kernel space.

File Metadata

- **Metadata** is data about data, in this case file data
- **Per-file metadata maintained by kernel**
 - accessed by users with the **stat** and **fstat** functions

```
/* Metadata returned by the stat and fstat functions */
struct stat {
    dev_t      st_dev;      /* device */
    ino_t      st_ino;      /* inode */
    mode_t     st_mode;     /* protection and file type */
    nlink_t    st_nlink;    /* number of hard links */
    uid_t      st_uid;      /* user ID of owner */
    gid_t      st_gid;      /* group ID of owner */
    dev_t      st_rdev;     /* device type (if inode device) */
    off_t      st_size;     /* total size, in bytes */
    unsigned long st_blksize; /* blocksize for filesystem I/O */
    unsigned long st_blocks; /* number of blocks allocated */
    time_t     st_atime;     /* time of last access */
    time_t     st_mtime;     /* time of last modification */
    time_t     st_ctime;     /* time of last change */
};
```

Example of Accessing File Metadata

```
/* statcheck.c - Querying and manipulating a file's meta data */
#include "csapp.h"

int main (int argc, char **argv)
{
    struct stat stat;
    char *type, *readok;

    Stat(argv[1], &stat);
    if (S_ISREG(stat.st_mode))
        type = "regular";
    else if (S_ISDIR(stat.st_mode))
        type = "directory";
    else
        type = "other";
    if ((stat.st_mode & S_IRUSR)) /* OK to read?*/
        readok = "yes";
    else
        readok = "no";

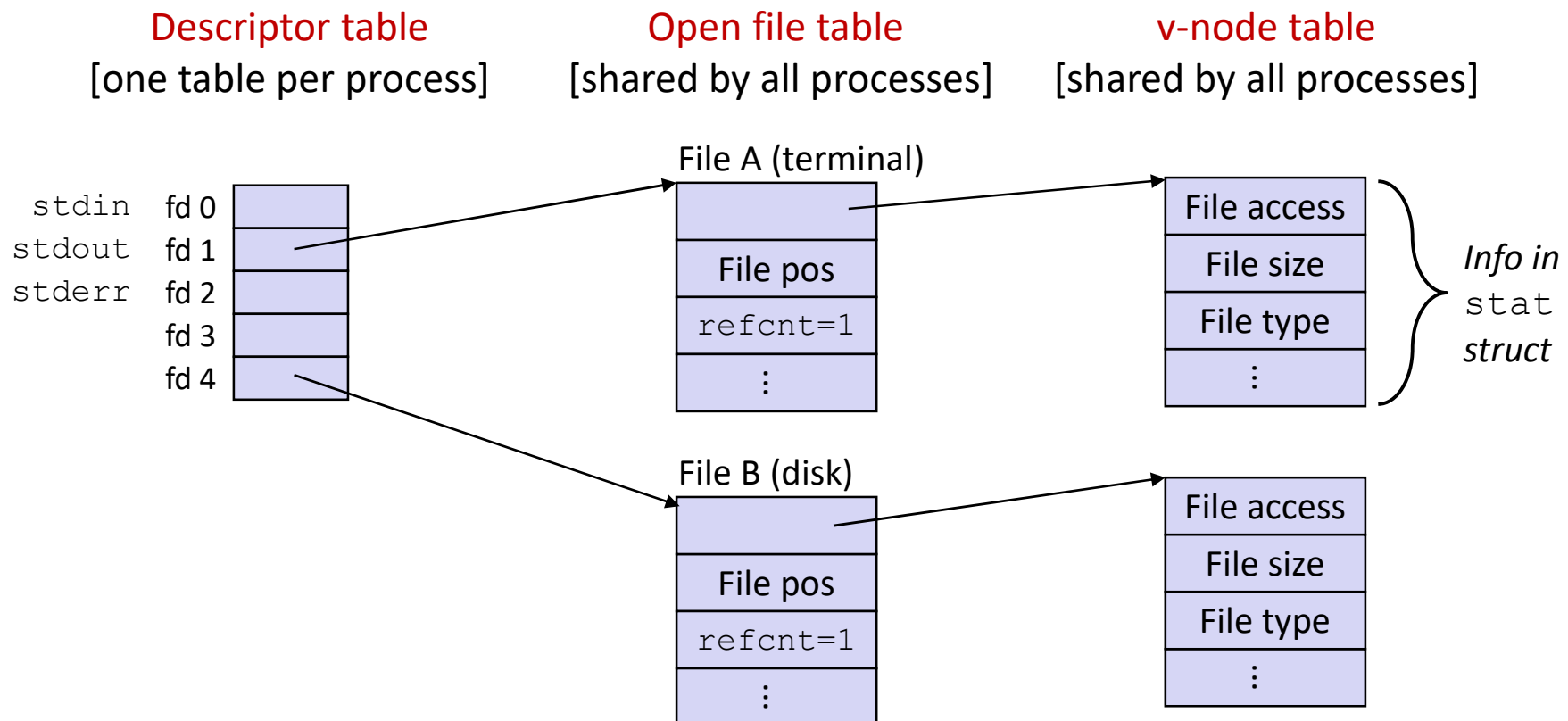
    printf("type: %s, read: %s\n", type, readok);
    exit(0);
}
```

```
unix> ./statcheck statcheck.c
type: regular, read: yes
unix> chmod 000 statcheck.c
unix> ./statcheck statcheck.c
type: regular, read: no
unix> ./statcheck ..
type: directory, read: yes
unix> ./statcheck /dev/kmem
type: other, read: yes
```

statcheck.c

How the Unix Kernel Represents Open Files

- Two descriptors referencing two distinct open disk files.
Descriptor 1 (stdout) points to terminal, and descriptor 4 points to open disk file



File Sharing and Redirection

- **File Sharing**

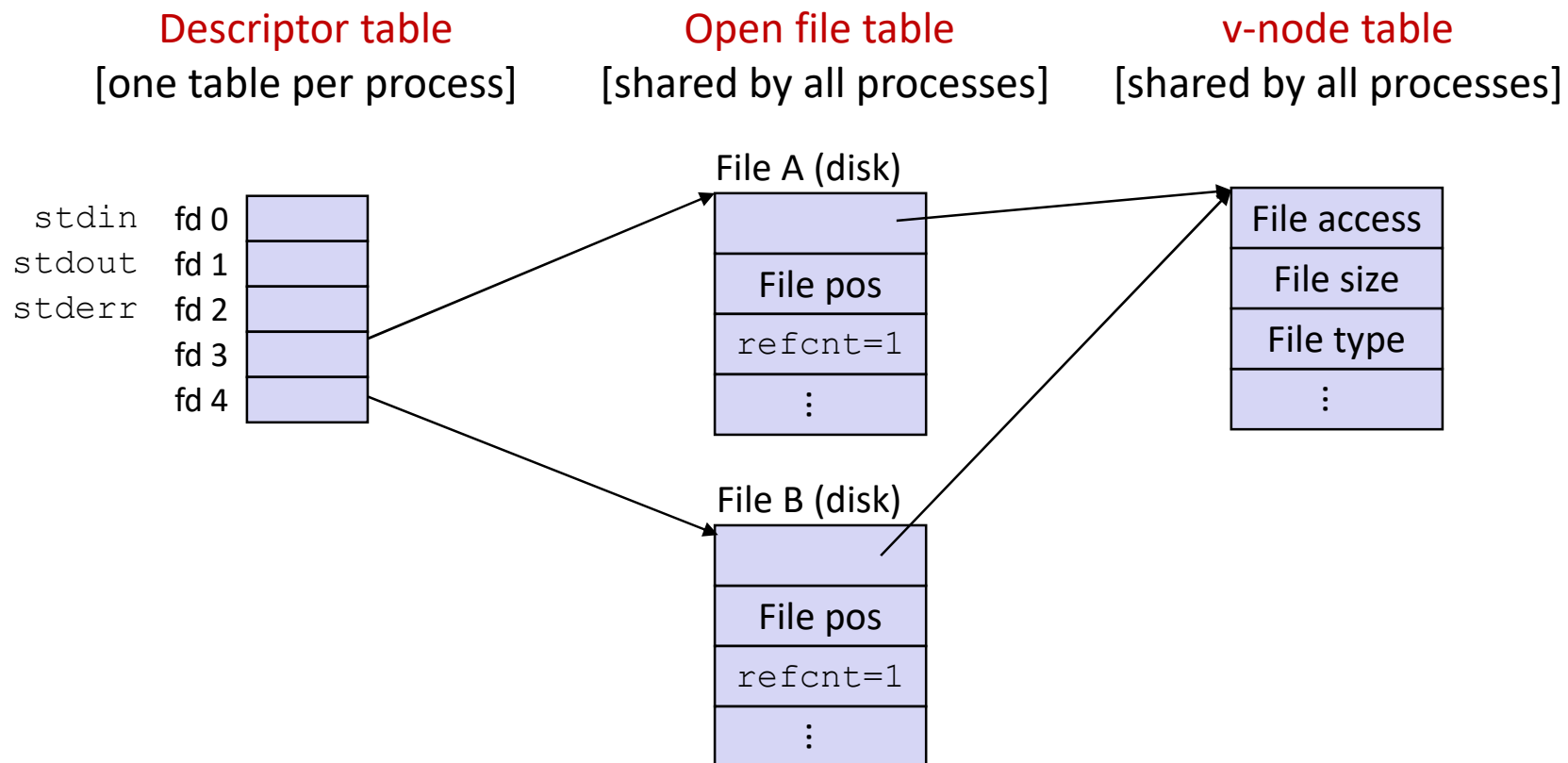
- **Unix:** Multiple processes can share an entry in the open file table (sharing the file offset) if a process is `fork()`-ed.
- **Windows:** Sharing is explicitly managed by "Share Modes" (`FILE_SHARE_READ`, `FILE_SHARE_WRITE`) during the `CreateFile` call.

- **I/O Redirection**

- **Unix:** Uses the `dup2(oldfd, newfd)` system call to copy file descriptors.
- **Mechanism:** To redirect `stdout` to a file, the shell closes FD 1 and `dup2`s the file's FD into slot 1.
- **Windows:** Accomplished via `SetStdHandle`.

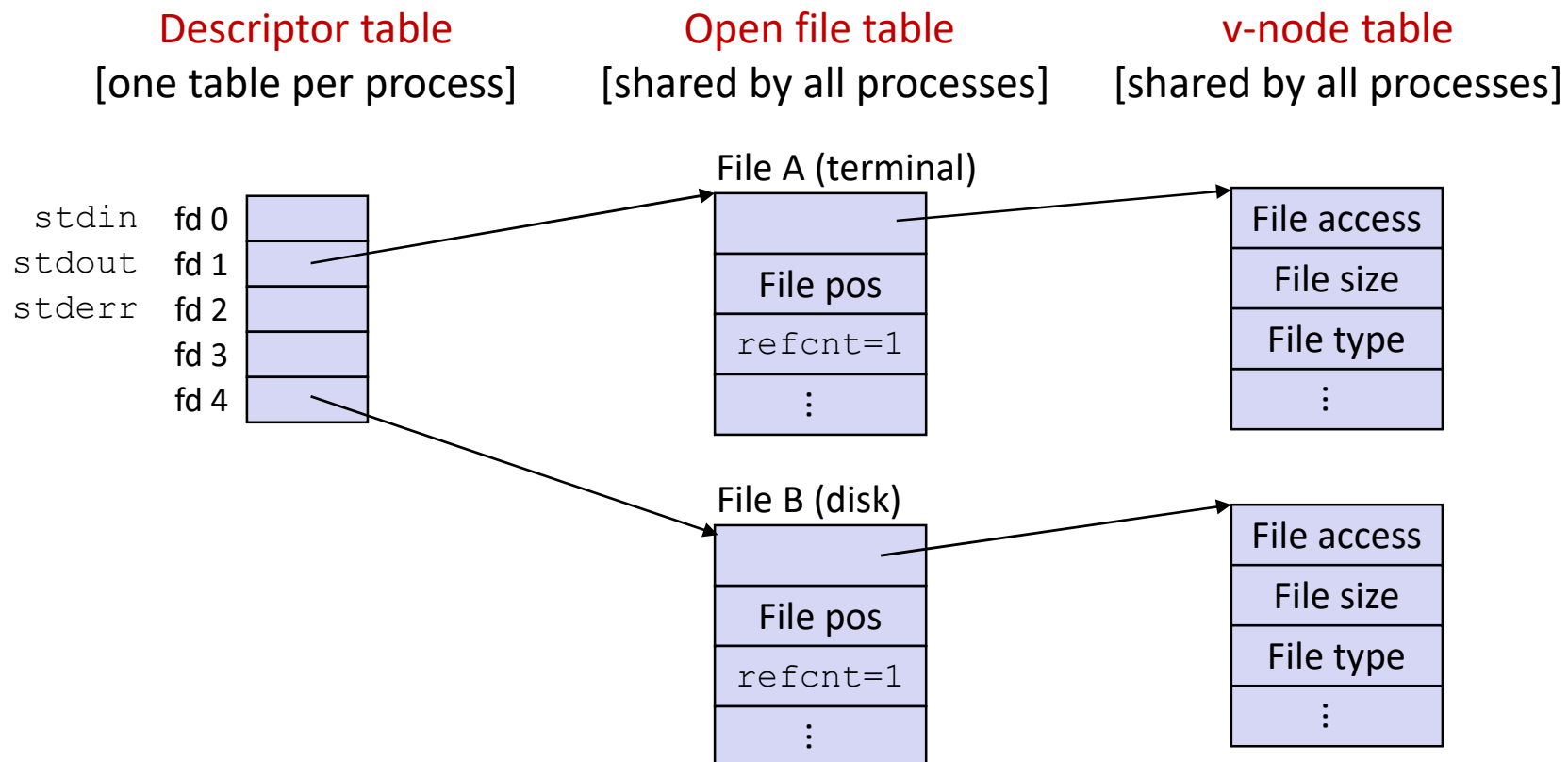
File Sharing

- Two distinct descriptors sharing the same disk file through two distinct open file table entries
 - E.g., Calling `open` twice with the same `filename` argument



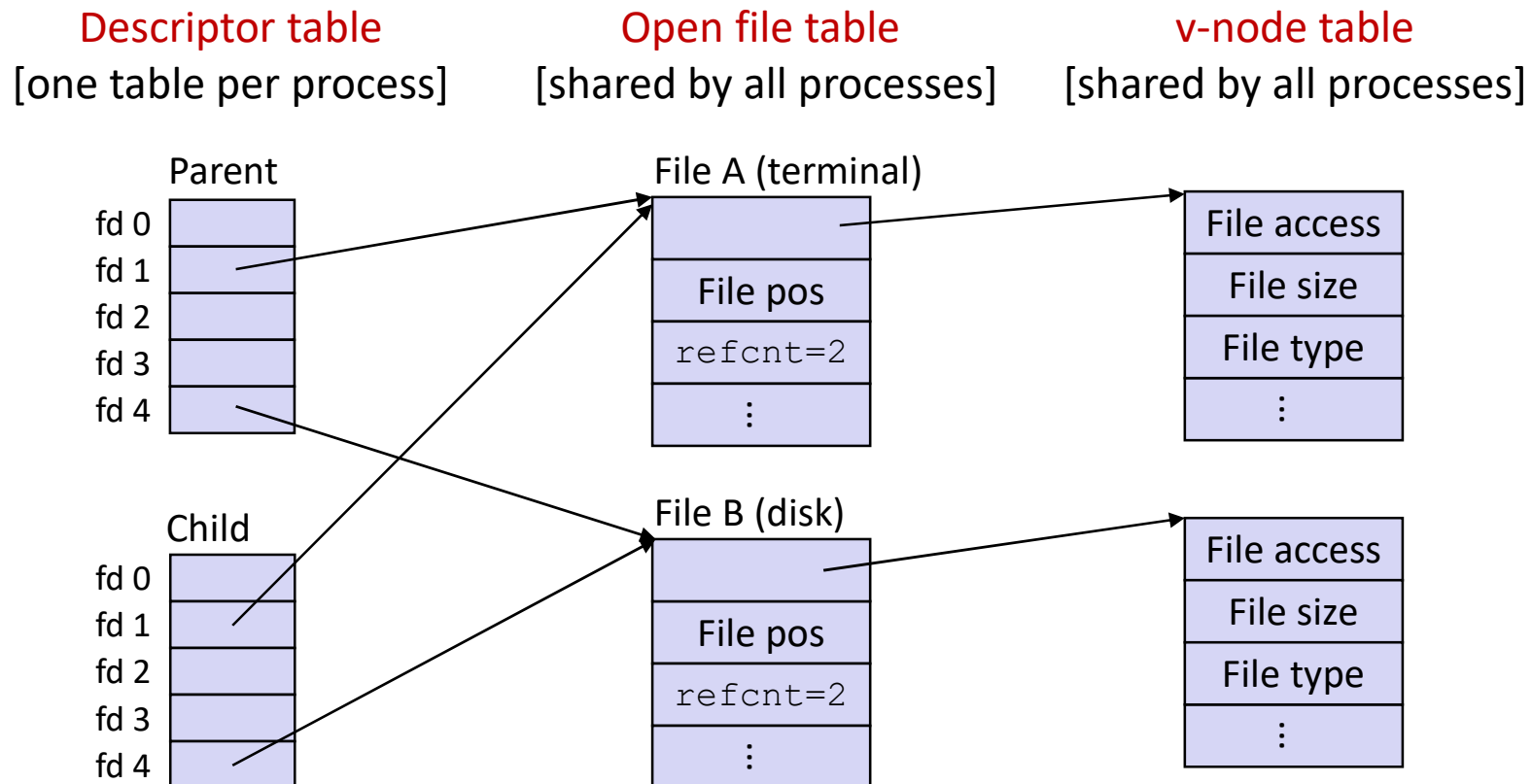
How Processes Share Files: Fork()

- A child process inherits its parent's open files
 - Note: situation unchanged by **exec** functions (use **fcntl** to change)
- **Before** fork() call:



How Processes Share Files: Fork()

- A child process inherits its parent's open files
- **After** fork():
 - Child's table same as parent's, and +1 to each refcnt



I/O Redirection

- Question: How does a shell implement I/O redirection?

```
unix> ls > foo.txt
```

- Answer: By calling the `dup2 (oldfd, newfd)` function
 - Copies (per-process) descriptor table entry `oldfd` to entry `newfd`

Descriptor table

before `dup2 (4, 1)`

fd 0	
fd 1	a
fd 2	
fd 3	
fd 4	b



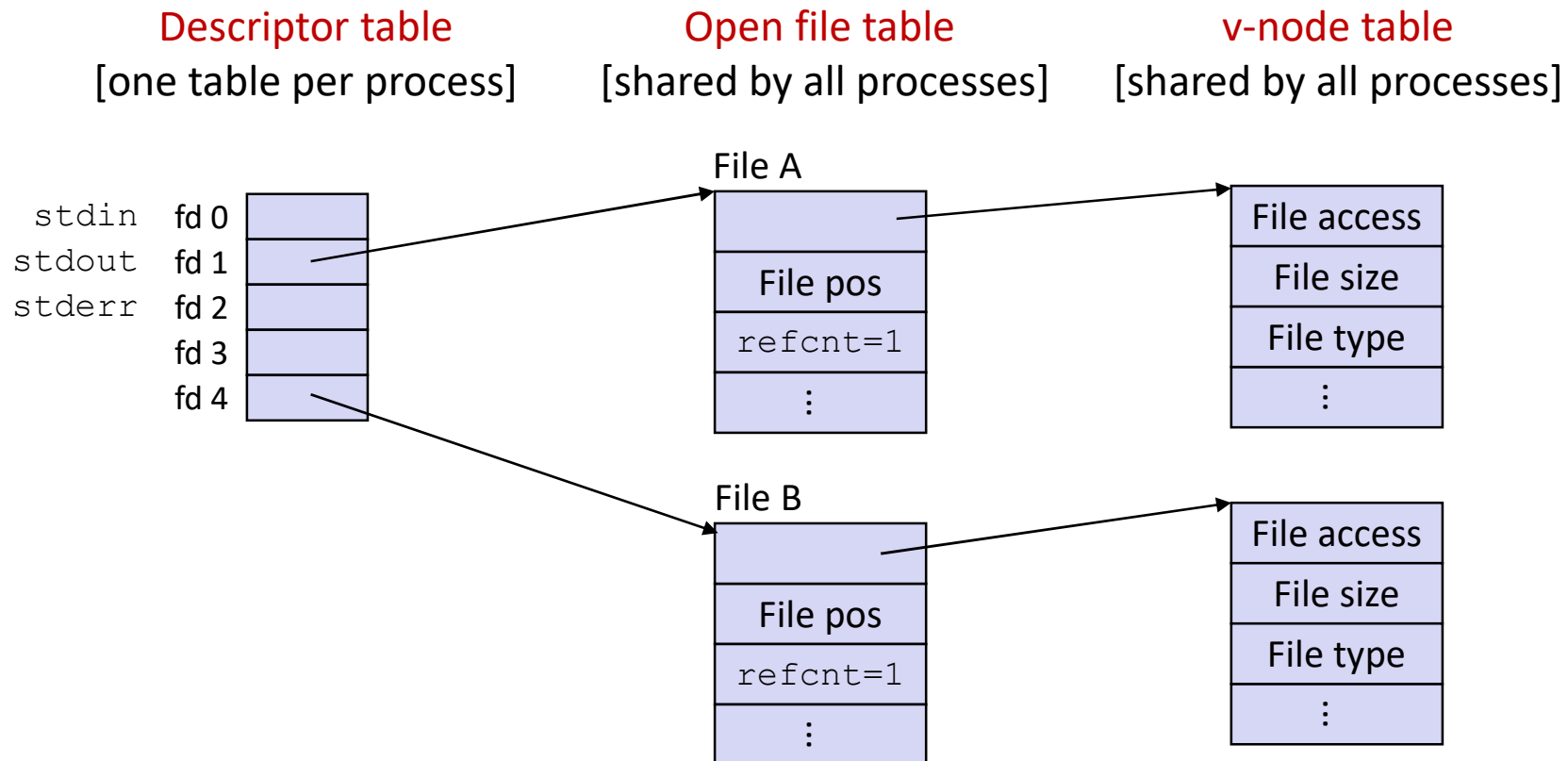
Descriptor table

after `dup2 (4, 1)`

fd 0	
fd 1	b
fd 2	
fd 3	
fd 4	b

I/O Redirection Example

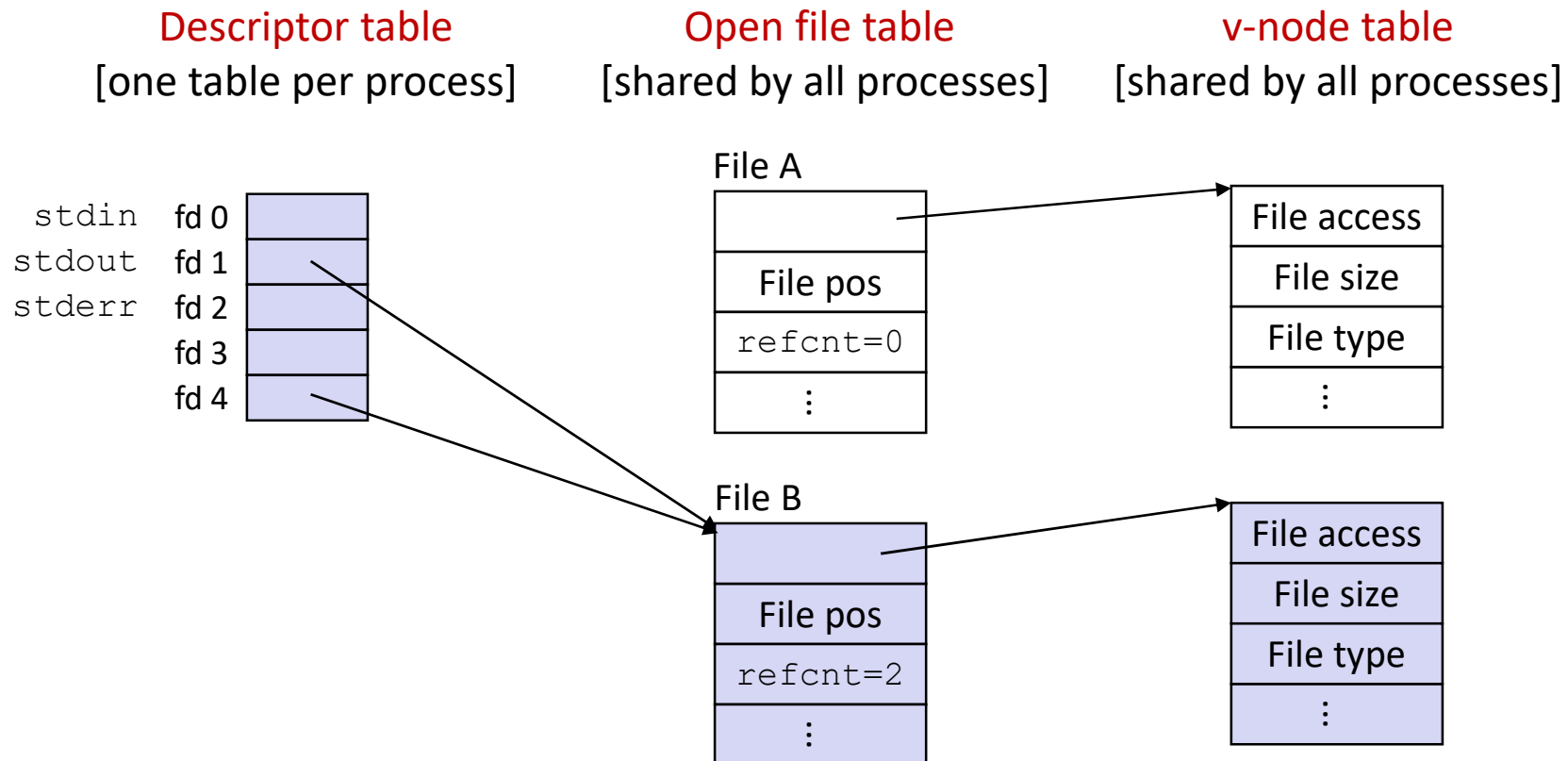
- **Step #1: open file to which stdout should be redirected**
 - Happens in child executing shell code, before **exec**



I/O Redirection Example (cont.)

■ Step #2: call `dup2 (4 , 1)`

- cause fd=1 (stdout) to refer to disk file pointed at by fd=4



Standard I/O, Streams, and Buffering

While the system calls are "Unbuffered I/O," most use the **Standard I/O Library (C standard library)**.

- **Streams:** An abstraction (`FILE *` in C) that wraps the raw file descriptor.
- **Buffered I/O:** To reduce expensive system calls, the library maintains a buffer in user memory.
 - **Full Buffering:** Writes only when the buffer is full (default for files).
 - **Line Buffering:** Writes on every newline `\n` (default for terminal/stdout).
 - **No Buffering:** Writes immediately (default for stderr).

Binary vs. Text Files

- **Unix:** No distinction. A file is a sequence of bytes.
- **Windows:** In "Text Mode," the library automatically translates `\n` (newline) to `\r\n` (carriage return + line feed) on write, and vice versa on read.

Standard I/O Functions

- The C standard library (`libc.so`) contains a collection of higher-level *standard I/O* functions
 - Documented in Appendix B of K&R
- Examples of standard I/O functions:
 - Opening and closing files (`fopen` and `fclose`)
 - Reading and writing bytes (`fread` and `fwrite`)
 - Reading and writing text lines (`fgets` and `fputs`)
 - Formatted reading and writing (`fscanf` and `fprintf`)

Standard I/O Streams

- Standard I/O models open files as *streams*
 - Abstraction for a file descriptor and a buffer in memory
- C programs begin life with three open streams (defined in `stdio.h`)
 - `stdin` (standard input)
 - `stdout` (standard output)
 - `stderr` (standard error)

```
#include <stdio.h>
extern FILE *stdin; /* standard input (descriptor 0) */
extern FILE *stdout; /* standard output (descriptor 1) */
extern FILE *stderr; /* standard error (descriptor 2) */

int main() {
    fprintf(stdout, "Hello, world\n");
}
```

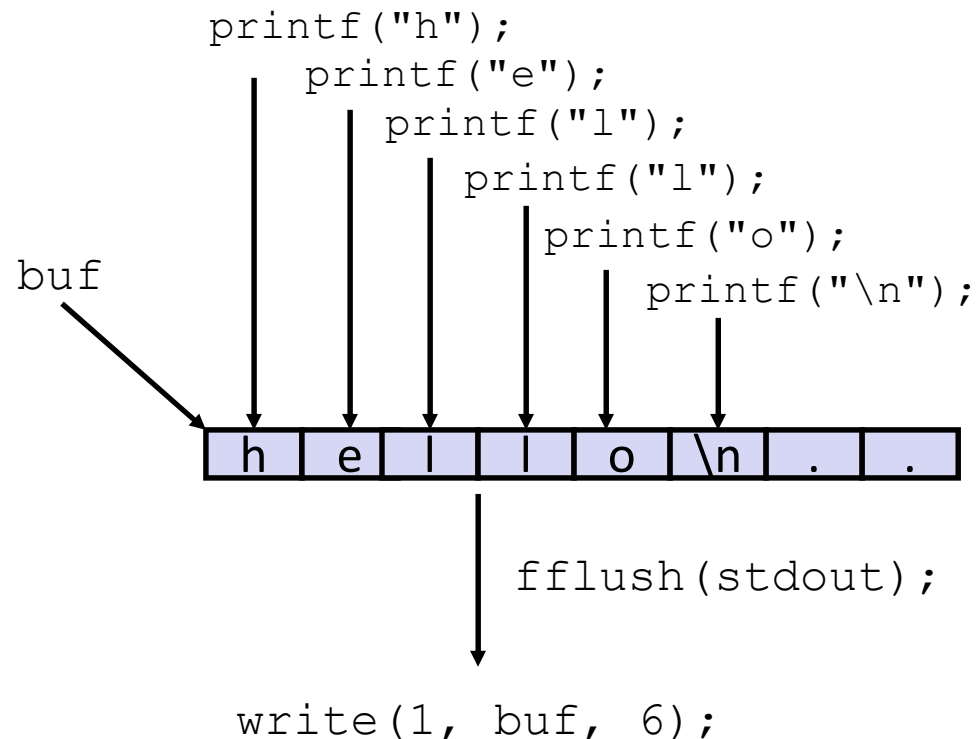
Buffered I/O: Motivation

- **Applications often read/write one character at a time**
 - `getc`, `putc`, `ungetc`
 - `gets`, `fgets`
 - Read line of text one character at a time, stopping at newline
- **Implementing as Unix I/O calls expensive**
 - `read` and `write` require Unix kernel calls
 - > 10,000 clock cycles
- **Solution: Buffered read**
 - Use Unix `read` to grab block of bytes
 - User input functions take one byte at a time from buffer
 - Refill buffer when empty



Buffering in Standard I/O

- Standard I/O functions use buffered I/O



- Buffer flushed to output fd on “\n” or `fflush()` call

Standard I/O Buffering in Action

- You can see this buffering in action for yourself, using the always fascinating Unix `strace` program:

```
#include <stdio.h>

int main()
{
    printf("h");
    printf("e");
    printf("l");
    printf("l");
    printf("o");
    printf("\n");
    fflush(stdout);
    exit(0);
}
```

```
linux> strace ./hello
execve("./hello", ["hello"], [/* ... */]).
...
write(1, "hello\n", 6)                = 6
...
exit_group(0)                         = ?
```

Pros and Cons of Unix I/O

■ Pros

- Unix I/O is the most general and lowest overhead form of I/O.
 - All other I/O packages are implemented using Unix I/O functions.
- Unix I/O provides functions for accessing file metadata.
- Unix I/O functions are async-signal-safe and can be used safely in signal handlers.

■ Cons

- Dealing with short counts is tricky and error prone.
- Efficient reading of text lines requires some form of buffering, also tricky and error prone.
- Both of these issues are addressed by the standard I/O and RIO packages.

Pros and Cons of Standard I/O

■ Pros:

- Buffering increases efficiency by decreasing the number of **read** and **write** system calls
- Short counts are handled automatically

■ Cons:

- Provides no function for accessing file metadata
- Standard I/O functions are not async-signal-safe, and not appropriate for signal handlers.
- Standard I/O is not appropriate for input and output on network sockets
 - There are poorly documented restrictions on streams that interact badly with restrictions on sockets (CS:APP2e, Sec 10.9)

Physical I/O Organization

How Devices are Represented

- **Unix:** Special files in `/dev/`. **Character devices** (keyboards) provide a stream of bytes; **Block devices** (disks) allow random access.
- **Windows:** Devices have internal names like `\Device\HarddiskVolume1`. Users access them via symbolic links like `C:` or `COM1`.

Physical Organization

- **Interrupts:** Hardware signals the CPU when an I/O operation is complete.
- **DMA (Direct Memory Access):** Allows hardware to transfer data directly to RAM without constant CPU intervention.
- **Device Drivers:** The bridge between the OS kernel and the specific hardware registers.

```
from multiprocessing import Process, Lock
```

```
def find_sum(x, l):
```

```
    s = 0
```

```
    for i in x:
```

```
        s += i
```

```
    l.acquire()
```

```
    print(s)
```

```
    l.release()
```

```
    return s
```

```
if __name__ == '__main__':
```

```
    lock = Lock()
```

```
    q1 = list(range(1, 10000, 2))
```

```
    q2 = list(range(2, 10000, 2))
```

```
    p1 = Process(target=find_sum, args=(q1, lock))
```

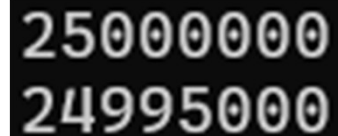
```
    p2 = Process(target=find_sum, args=(q2, lock))
```

```
    p1.start()
```

```
    p2.start()
```

```
    p1.join()
```

```
    p2.join()
```



25000000
24995000

```
from multiprocessing import Process, Pipe
```

```
def f(conn):
```

```
    x = conn.recv()
```

```
    for i in range(len(x)):
```

```
        x[i] += 5
```

```
    x.append(42)
```

```
    x.append('Hello')
```

```
    conn.send(x)
```

```
    conn.close()
```

```
[61, 32, 42, 'Hello']
```

```
if __name__ == '__main__':
```

```
    parent_conn, child_conn = Pipe()
```

```
    p = Process(target=f, args=(child_conn,))
```

```
    p.start()
```

```
    parent_conn.send([56, 27])
```

```
    print(parent_conn.recv())
```

```
    p.join()
```